CS 486/686
Pretraining Language Models From First Principles

Yuntian Deng

Lecture 20

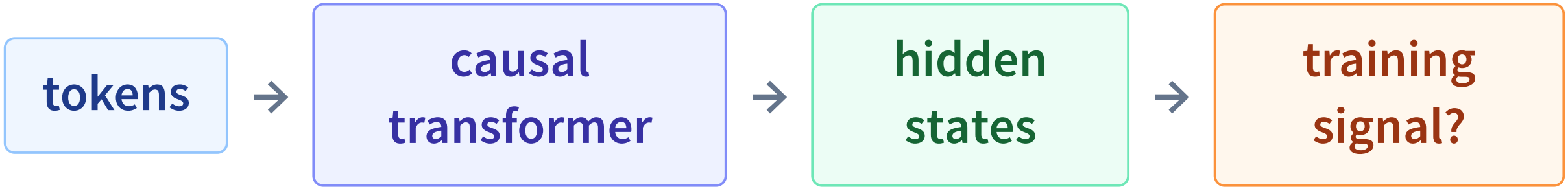
How raw text becomes a training signal

Search › Uncertainty › Decisions › Learning

By the end, you can trace one training example

- Turn text into **tokens, IDs, and vectors**.
- Build **next-token targets** from one sequence.
- Trace $\mathbf{h}_t \longrightarrow$ logits $\longrightarrow$ probability $\longrightarrow$ loss.
- Explain how one batch **updates every parameter**.
- Explain why targets are automatic but **data curation is not**.
- Contrast parallel **training** with sequential **generation**.

L19 built the machine. What teaches it?



Give it one task, billions of times: **predict what comes next.**

A language model predicts a distribution

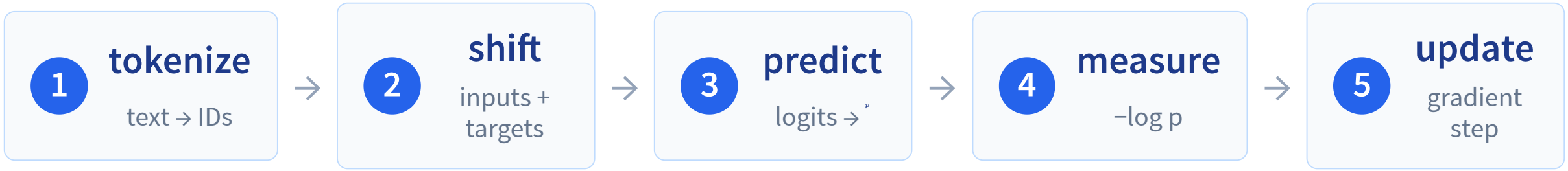
`The robot picked up the cup because it was ___`

empty	**blue**	**broken**	**quantum**
very plausible	also plausible	possible	unlikely

training text continues with **empty**

The label is the *observed* next token—not the only meaningful continuation.

Our roadmap: text becomes a gradient



Tokens are not words

`hello` ≠ `·hello`

a leading space matters

`CS486` → `CS | 4 | 8 | 6`

rare strings split

`unbelievable` → `un | belie | vable`

pieces need not be morphemes

🤖 □□ → `tokens`

bytes let any Unicode text be represented

Token count controls sequence length—and therefore memory and compute.

Learn reusable pieces by merging frequent pairs

start with characters	`l | o | w | e | r`	`l | o | w | e | s | t`
merge l + o	`lo | w | e | r`	`lo | w | e | s | t`
after many merges	`low | er`	`low | est`

Save the learned pieces: the tokenizer can now emit **`low`**, **`er`**, **`est`**, … as individual tokens.

Freeze this piece list, then use the same tokenizer for every language-model example.

Byte-pair encoding intuition; Sennrich, Haddow & Birch, “Neural Machine Translation of Rare Words with Subword Units,” 2016.

A real tokenizer, piece by piece LIVE

Qwen3 tokenizer · 151,669 entries · · means “this token begins with a space.”

unbelievable | CS486 | hello | ·hello | 🤖 ⊠⊠ | code

unbelievable

tokenize custom text

un	belie	vable
359	31798	23760

12 characters **12** UTF-8 bytes **3** tokens

Separate chips are separate tokens. The · symbol is not a separator; it represents one leading blank.

IDs retrieve learned vectors

token	ID	lookup
`The`	785	$\mathbf{E}_{785}$
`·robot`	12305	$\mathbf{E}_{12305}$
`·empty`	4287	$\mathbf{E}_{4287}$

IDs are addresses.
Nearby numbers need not mean nearby concepts.

Embedding rows are learned.
Pretraining gives the vectors meaning.

embedding vectors with **positional information** → **causal transformer**

Actual IDs from the Qwen3 tokenizer; vocabulary size 151,669.

Shift once: one sequence gives many labels

Five observed tokens become five classification labels—no manual annotation.

Teacher forcing predicts all positions together

position	1	2	3	4	5
given token	`The`	`robot`	`picked`	`up`	`the`
predict	`robot`	`picked`	`up`	`the`	`cup`
may read	1	1–2	1–3	1–4	1–5

One forward pass: all true prefixes are present; the causal mask blocks every future token.

Each hidden state votes over the vocabulary

token IDs $[B, T]$ → **transformer** $\mathbf{H} : [B, T, d]$ → **output head** $\boldsymbol{\ell} : [B, T, V]$

$$\boldsymbol{\ell}_t = \mathbf{W}_{\text{out}} \mathbf{h}_t + \mathbf{b}$$

At every position: V scores—one for every possible next token.

Logits are unnormalized preferences

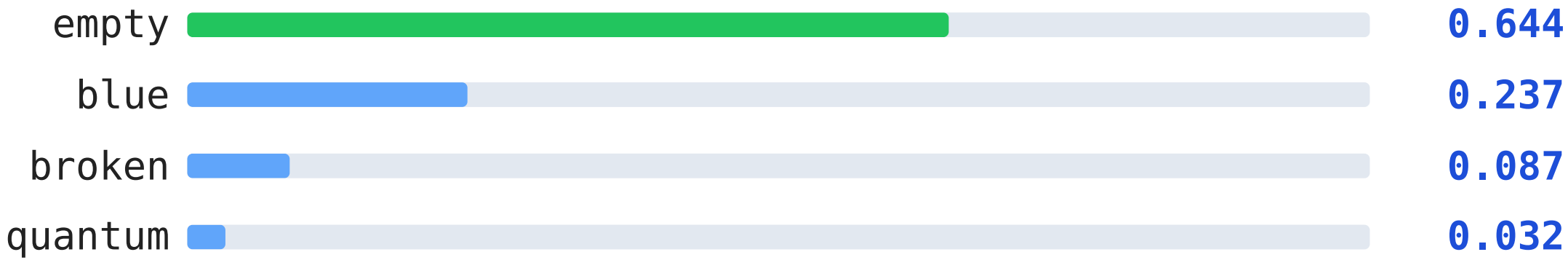
TOY VOCABULARY context: `The robot picked up the cup because it was`

candidate	logit	meaning
`empty`	**2.0**	observed target
`blue`	**1.0**	second choice
`broken`	**0.0**	less preferred
`quantum`	**−1.0**	least preferred

Logits can be negative and need not sum to anything.

Softmax turns those scores into probabilities

$$p_i = \frac{e^{\ell_i}}{\sum_j e^{\ell_j}}$$



$$0.644 + 0.237 + 0.087 + 0.032 = 1.000$$

The observed token determines the loss

observed next token

empty

model assigned

$$p(\text{empty}) = 0.644$$

cross-entropy

$$-\ln 0.644 = 0.44$$

$$p(\text{target}) = 0.90 \Rightarrow L = 0.11$$

$$p(\text{target}) = 0.01 \Rightarrow L = 4.61$$

Confident and wrong is expensive.

A text probability is built left to right

x_1 x_2 x_3 $\dots$ x_T

$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=1}^{T-1} P(x_{t+1} \mid x_{\leq t})$$

$P(\text{The})$ × $P(\text{robot} \mid \text{The})$ × $P(\text{picked} \mid \text{The robot})$ ×...

The same conditional probabilities power both training and generation.

Training averages the token losses

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid \text{context}_i)$$

N

non-padding targets in the batch

lower is better

on held-out validation text

perplexity

$\exp(\mathcal{L}_{\text{val}})$

If $\mathcal{L}_{\text{val}} = 2$, perplexity is $e^2 \approx 7.4$: uncertainty like roughly seven equally likely choices.

Inspect a real model's loss LIVE DATA

Qwen3-0.6B-Base · click a target token to inspect its prefix, probability, and loss.

Color encodes target loss: green = lower, amber = medium, red = higher. Numbers are nats.

natural continuation | surprising continuation

·robot	·picked	·up	·the	·cup	·because	·it	·was	·empty
10.7	8.3	0.4	1.4	5.1	6.9	0.8	0.8	3.7

.
0.9

prefix
`The robot picked`

target ·up p = **0.668** −ln p = **0.40** predictable here → small gradient signal

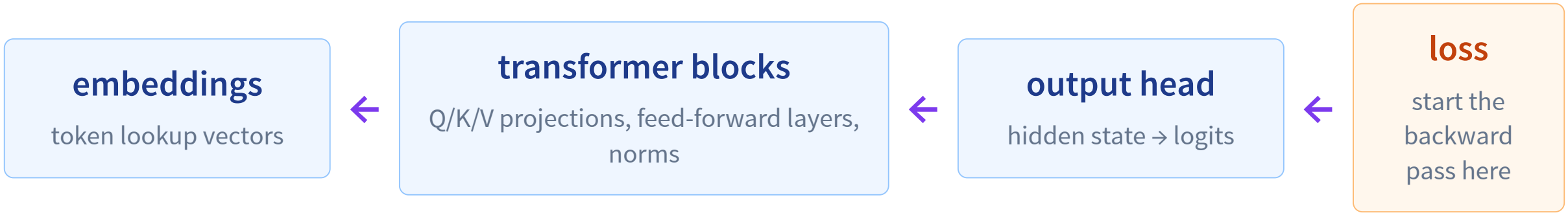
average NLL **3.90** perplexity **49.2** **10** training signals

Click any target token. Its prefix is exactly what that position could use under the causal mask.

The pretraining loop, line by line

```
# packed token sequences: one extra token supplies the final target
ids = next(token_batches)                # [B, T+1]
# every position predicts the token immediately to its right
x, y = ids[:, :-1], ids[:, 1:]           # each [B, T]
# one vocabulary-sized score vector at every position
logits = model(x).logits                  # [B, T, V]
# average next-token cross-entropy over the whole batch
loss = cross_entropy(logits.reshape(-1, V), y.reshape(-1))
# discard gradients left from the previous update
optimizer.zero_grad()
# autograd sends credit and blame through the entire model
loss.backward()
# update all trainable parameters once
optimizer.step()
```

The gradient reaches the whole model



Important: attention *weights* are recomputed for each input; the projection matrices that produce them are learned parameters.

Self-supervised does not mean “free”

Targets are automatic

`The robot picked` → `up`

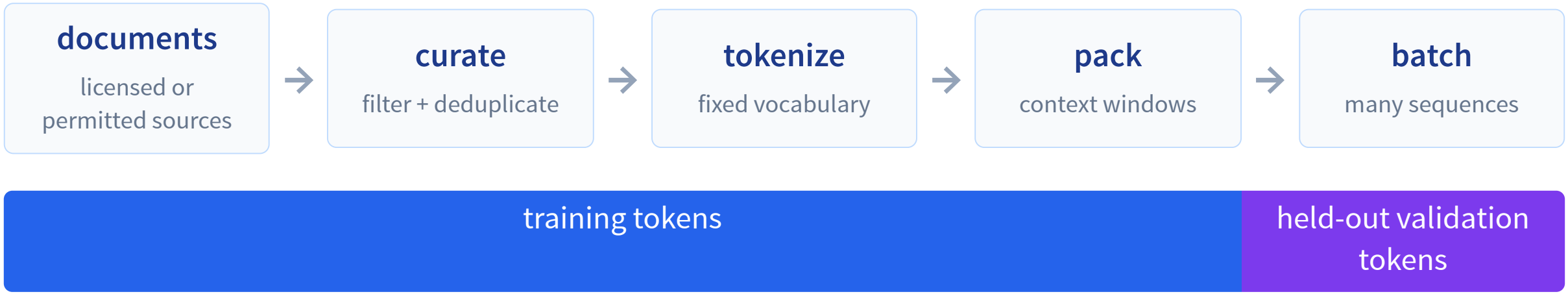
No person labels every next token.

Data is not automatic

- permission and licensing
- filtering and deduplication
- privacy, safety, and quality

The objective is cheap to label; the corpus is costly to build well.

Before training: build the token stream



Never evaluate next-token loss on text used for parameter updates.

Scale is a budget: balance model and data

too many parameters

too few tokens to train them well

balanced for fixed compute

parameters and tokens grow together

rough training compute $\propto$ parameters $\times$ training tokens

Hoffmann et al., “Training Compute-Optimal Large Language Models,” 2022.

What can next-token prediction teach?

syntax + style	`The keys are on the table.`
associations	`Waterloo is in Ontario.`
procedures	`for i in range(3): print(i)`

To predict well, hidden states must capture
many recurring patterns in text and code.

But the objective guarantees only prediction

truth

frequent text can still be false

reasoning

pattern completion can fail off-distribution

instructions

a base model is trained to continue, not obey

recency

weights do not know events after training

privacy

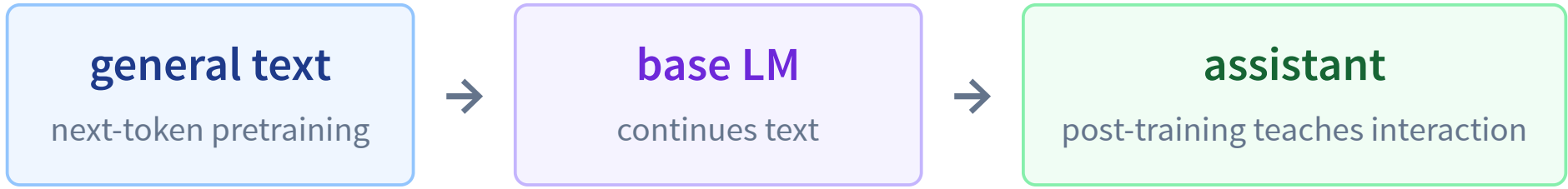
training text may be memorized

new context

private/current facts must be supplied safely

Low loss is useful—not a certificate of truth or helpfulness.

Pretraining creates a base language model



`Explain gradient descent:` a base model may continue the document—not necessarily follow the request.

Later: instruction tuning, preference optimization, prompting, RAG, and tools.

Same model, different execution

training

true sequence → all positions → loss → update weights

- true prefixes supplied
- many token predictions in parallel

generation

prompt → one distribution → choose + append ↺

- weights fixed
- new tokens arrive sequentially

Choose one token, append, repeat REAL MODEL DATA

Qwen3-0.6B-Base · choose a rule, then click repeatedly to grow the context one token at a time.

Shakespeare | robot | story

step 1: **To be, or not to** → ?

·be	0.991
↵	0.001
,	0.001
·have	4.3e-4
·Be	4.3e-4
be	3.3e-4
all other vocabulary tokens	0.004

highest each step | sample each step

append next token | reset

Click “append next token” repeatedly. Each click uses the distribution conditioned on the enlarged context.

```
To be, or not to
```

After every append, the bars update to the next real conditional distribution. L21 adds temperature, top-k, and top-p.

Next: watch Qwen run from the inside

inspect
tokenizer, weights, attention

generate
append loop + KV cache

control
greedy, sampling, temperature, top-k/top-p

condition
chat templates and prompts

L21: Dissecting Qwen3-0.6B.

Exit check: can you trace the signal?

The	**robot**	**picked**
robot	**picked**	**up**

At the final position, the model assigns $p(\text{up}) = 0.80$.

target up

token loss $-\ln 0.80 \approx 0.22$

training average with other positions, then backpropagate

generation choose a token, append it, repeat

Text → targets → probabilities → loss → gradients.